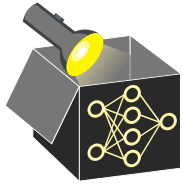
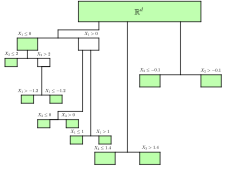
Interpretable Machine Learning



Interpretable Models
Random Planted Forests

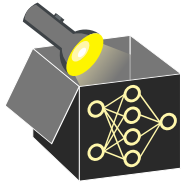


Learning goals

- Motivation for RPFs
- Understand node types and restricting interactions in decision trees
- Understand planted trees: non-binary decision trees and inner leaves

RANDOM PLANTED FORESTS (RPF)

▸ "Hiabu et al." 2023

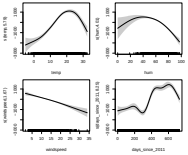


Goal: Create a powerful tree ensemble, but still interpretable

Idea:

- GAMs easily interpretable, because no interaction $\rightsquigarrow$ plot 1D functions

$$\hat{f}(x) = \theta_0 + f_1(x_1) + f_2(x_2) + \ldots + f_p(x_p)$$



- Same for functions containing interactions between max. 2 features
  $\rightsquigarrow$ function of 2 features, plot 2D functions (i.e. 3D plot)

$$\hat{f}(x) = \theta_0 + f_1(x_1) + f_2(x_2) + \ldots + f_p(x_p) + f_{1,2}(x_1, x_2) + \ldots + f_{1,p}(x_1, x_p) + \ldots + f_{p-1,p}(x_{p-1}, x_p)$$

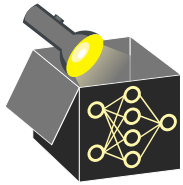
  $\rightsquigarrow$ Visualize single functions f_1, f_2, $f_{1,2}(x_1, x_2)$, $f_{1,3}(x_1, x_3)$, ...

$\Rightarrow$ Interpretability possible via restricting degree of interactions

Problem: How to know degree of interactions?

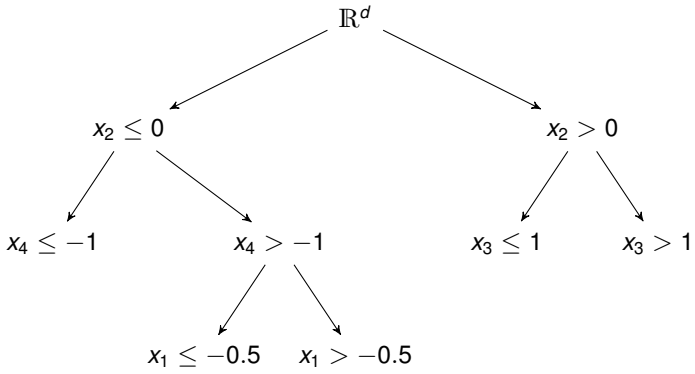
Solution: Easy to determine for trees / tree ensembles!

RPF: DETERMINE INTERACTION TYPE IN TREES

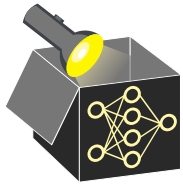


Define the *interaction type t* of a node as the subset of features involved in constructing this node.

Example:

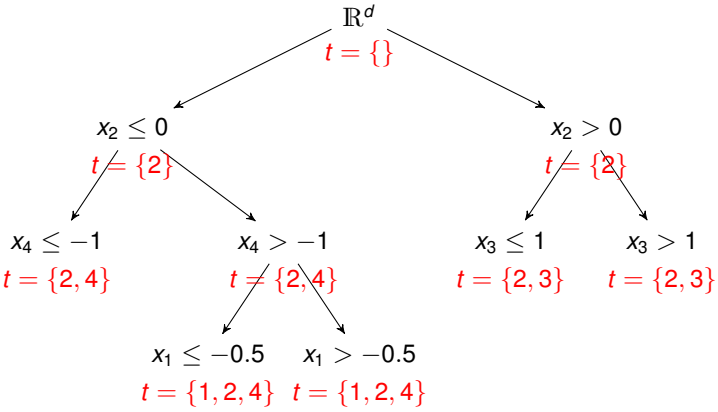


RPF: DETERMINE INTERACTION TYPE IN TREES



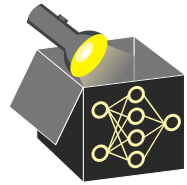
Define the *interaction type* t of a node as the subset of features involved in constructing this node.

Example:



$\Rightarrow$ Degree of interaction in each node is $|t|$.

RPF: BOUNDED INTERACTION ORDER + PLANTED TREES



Goal: restrict this interaction degree
$\rightsquigarrow$ In RPFs:

- Always keep track of the interaction type in each node
- For each new split, make sure the max. degree of interactions is not exceeded
  $\Rightarrow$ When max. number of feat reached, no new feat are allowed

Problem: For small interaction order, single trees quickly become limited

- E.g. interaction order 1: every tree uses only one feature

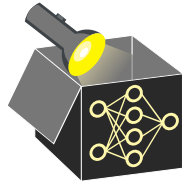
$\Rightarrow$ Many trees are needed for a more complex model

Idea: Allow inner nodes to split again

Define *Planted Trees*: decision trees where each inner node can be a *leaf*:

- Add prediction to final output
- Can be split again $\Rightarrow$ several splits possible

RPF: EXAMPLE



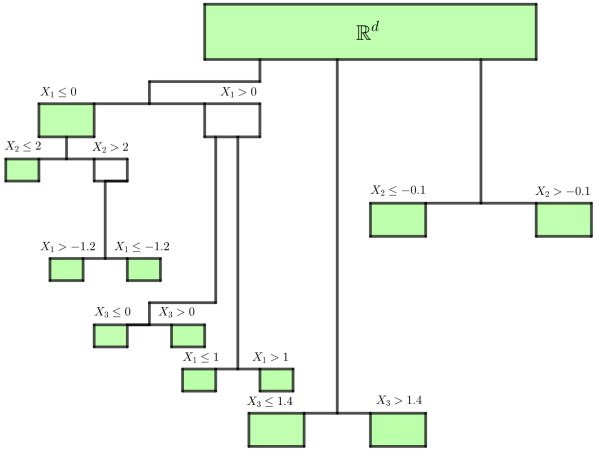
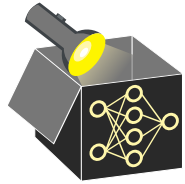


Figure: Example of a single fully grown planted tree, green nodes: “leaves”

RPF: EXAMPLE



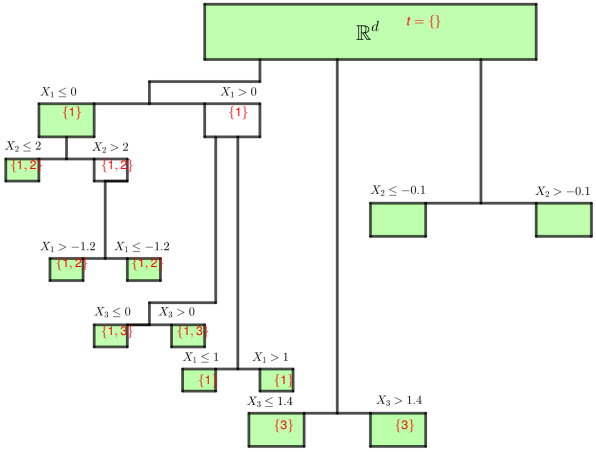
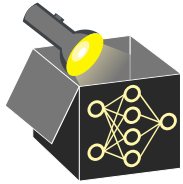


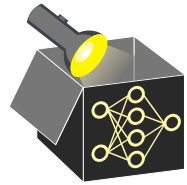
Figure: Example of a single fully grown planted tree, green nodes: “leaves”

RPF: ALGO



- Max. interaction degree is a hyperparameter
- Total number of trees is a hyperparameter
- End growing tree after the maximum total number of splits instead of the maximum depth
  (Minimum number of samples is also possible, but then higher nodes would split too often)
- Randomization as in random forests:
  - Only optimize over a randomly chosen subset of features
  - Only optimize over a subset of possible split values
- Make an inner leaf an inner node (i.e. delete the “leaf” property) if it has children with the same type

RPF: EXAMPLE RESULTS AND INTERPRETATION



RPF: CONCLUSION

